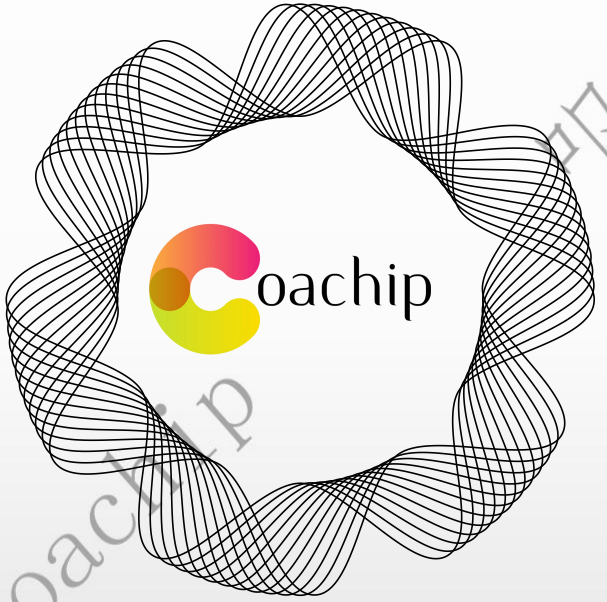


System Verilog
语言简介

叩特电子Coachip

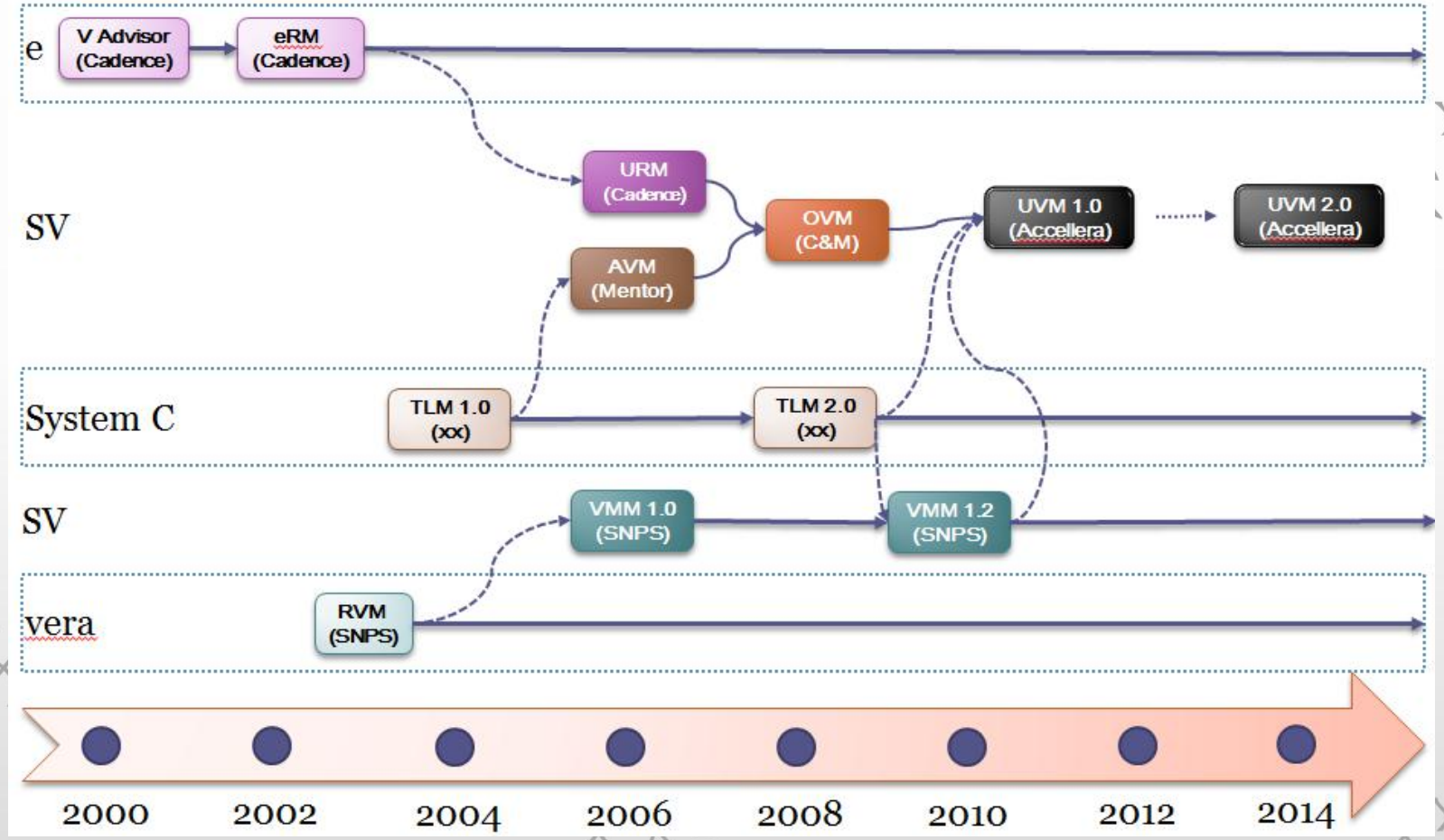
课程目标

- 了解SV作为验证语言的使用

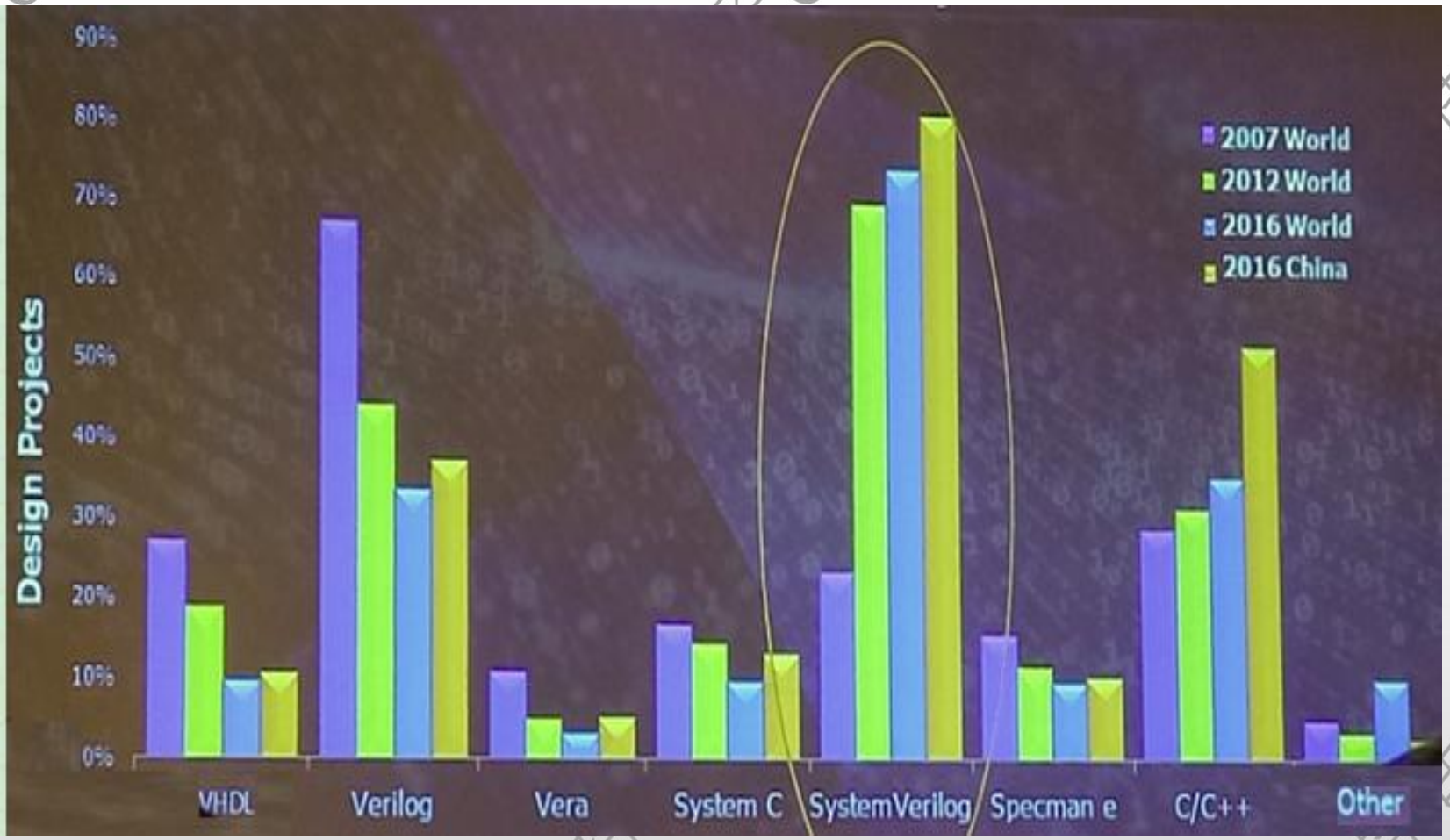
叩特电子Coachip

SV UVM

SV： 可以用于验证的语言　　UVM：用于验证的方法学



SystemVerilog Adoption in World

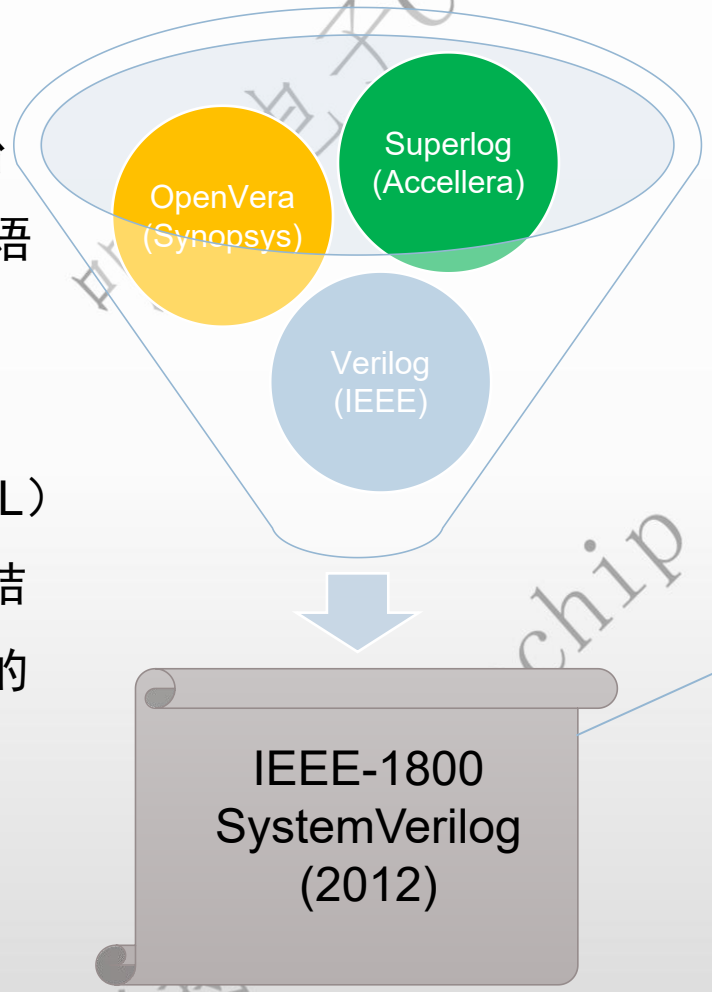


即特电子Coachip

SV语言

SystemVerilog结合了来自 Verilog、VHDL、C++的概念，还有验证平台语言（OpenVera和e语言）和断言语言（OVA和PSL）

SytemVerilog将硬件描述语言（HDL）与现代的高层级验证语言（HVL）结合了起来，对于进行当今高度复杂的设计验证具有相当大的吸引力



- Verilog
- OOP
- Assertions
- Coverage
- New data types
- Interfaces
- Random Generation
- DPI
- Synchro IPC

数据类型

- 内建类型（基本类型）
- 队列
- 数组
- 结构体（数据的集合）
- 枚举类型
- 字符串

Type	2-4 states	Signed/ Unsigned	Size (bits)	SV/V
shortint	2	signed	16	SV
int	2	Signed	32	SV
longint	2	Signed	64	SV
byte	2	Signed	8	SV
bit	2	Unsigned		SV
logic	4	Unsigned		SV
reg	4	Unsigned		V
net	4	Unsigned		V
integer	4	Signed	32	V
real				V
shortreal				SV
realtime				V
time	4	unsigned	64	V

叩特电子Coachip

类和对象（OOP面向对象）

那么验证世界中，为什么要有类的存在呢？

激励生成器（stimulus generator）：生成激励内容

驱动器（driver）：将激励以时序形式发送至DUT

监测器（monitor）：监测信号并记录数据

比较器（checker）：比较数据

验证环境的不同组件其功能和所需要处理的数据内容是不相同的

不同环境的同一类型的组件其所具备的功能和数据内容是相似的

基于以上两点，验证世界的各个组件角色明确、功能分立，使用面向对象编程与验证世界的构建原则十分符合。

叩特电子Coachip

类和对象（OOP面向对象）

class declaration（声明）

class members（成员）

class constructor（构造函数）

class methods（方法）

```
class wishbone_data extends ovm_object;
    item_type_t      item_type;  ///<! type of the current item
    rand bit [31:0] address;     ///<! wishbone address
    rand bit [31:0] data;        ///<! wishbone read/write data
    rand bit        wr_en;       ///<! write (1'b1) or read (1'b0)

    function new(string name="");
        super.new(name);
    endfunction : new

    function bit[31:0] get_address();
        return this.address;
    endfunction : get_address

    function set_address(bit [31:0] new_value);
        this.address=new_value;
    endfunction : set_address

endclass : wishbone_data
```

方法

- function

```
function int double(input a);
   return 2*a;
endfunction
```

- task

task内可以置入耗时语句，而function则不能。

常见的耗时语句包括@event、wait event、# delay等

叩特电子Coachip

过程块

- **initial**
  - initial非常符合软件的执行方式，即只执行一次
  - initial本身不可综合
  - initial过程块可以在module、interface和program中使用

- **always**
  - always过程块是用来描述硬件时序电路和组合电路的正确打开方式
  - 只可以在module或者interface中使用

注：SV中怎么区分硬件世界和软件世界呢？

module/endmodule，interface/endinterface可以被视为硬件世界

program/endprogram和class/endclass可以被视为软件世界。

叩特电子Coachip

IPC（Inter Process Communications）

- SystemVerilog Events

```
event trans_ack_evt;

initial begin
    fork
        task1();
        task2();
    join_none;
end

task task1();
    forever begin
        @(posedge clk);
        if(ack=1)begin
            -> trans_ack_evt;
        end
    end
endtask : task1

task task2();
    forever begin
        @(trans_ack_evt);
        $display("Task1 has detected an ACK for crt frame");
    end
endtask : task2
```

SV event declaration

Each time the signal ack is 1, we trigger the SV event "*trans_ack_evt*"

Block the process until the event is triggered by task1()

IPC（Inter Process Communications）

- mail box

```
mailbox my_mailbox;
initial begin
    my_mailbox = new();
    if (my_mailbox) begin
        fork
        store_transactions();
        get_transactions();
        join_none;
    end
end

task store_transactions();
    wishbone_trans crt_trans;
    forever begin
        @(posedge trans_received);
        my_mailbox.put(crt_trans);
    end
endtask : store_transactions

task get_transactions();
    wishbone_trans crt_trans;
    forever begin
        my_mailbox.get(crt_trans);
        crt_trans.print();
    end
endtask  : get_transactions
```

Create mailbox

Each time a transaction is received, we store in the mailbox

Blocking: wait here until a message is available

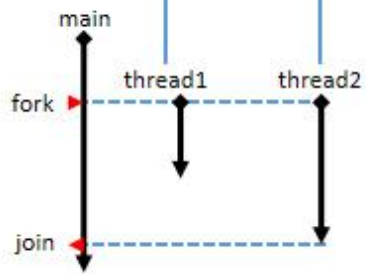
叩特电子Coachip

线程控制

FORK are used to launch tasks in parallel: threads.

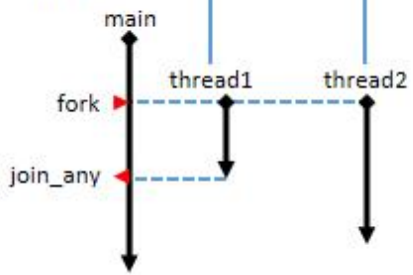
3 Types of fork:

```
fork
  press_power_button(800);
  press_user_button(800);
join;
```



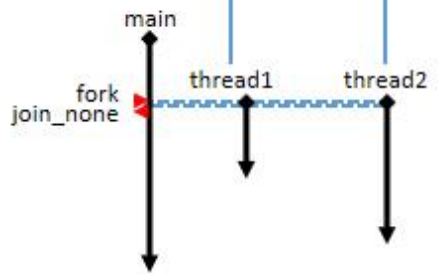
join: blocking statement. wait for all threads to complete

```
fork
    begin
      proces_frame();
    end
    begin
      @(posedge perform_reset_ovc);
    end
join_any;
```



join_any: blocking statement. wait for one thread to complete

```
fork
  collect_transactions();
  perform_checks();
join_none;
```



Join-none: non-blocking statement.doesn't wait on anything

随机激励产生

```
class order;
  rand bit [7:0] lo, med, hi;
  constraint bad
  {
    lo < med < hi;
  }
endclass
```

```
rand int c; // Random variable
int lo, hi; // Non-random variables
constraint c_range
{
  c inside {[lo:hi]}; // lo <= c && c <= hi
}
```

```
rand int src, dst;
  constraint c_dist
  {
    src dist {0:=40, [1:3]:=60};
    // src = 0, weight = 40/220
    // src = 1, weight = 60/220
    // src = 2, weight = 60/220
    // src = 3, weight = 60/220

    dst dist {0:/40, [1:3]:/60};
    // dst = 0, weight = 40/100
    // dst = 1, weight = 20/100
    // dst = 2, weight = 20/100
    // dst = 3, weight = 20/100
}
```

接口

Interface declaration

Signals of the interface

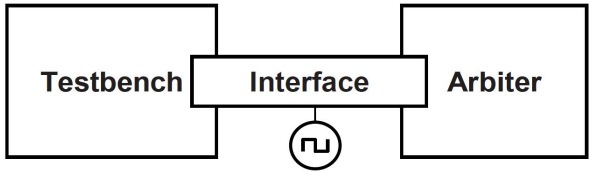
Can contain clocking blocks:
Encapsulate timing constraints

```
interface wishbone_interface;
    logic clk;
    logic rst_n;

    logic internal_clk;

    logic [31:0]    data_w;
    logic [31:0]    data_r;
    logic [31:0]    address;
    logic           wr_en;
    logic           strobe;
    logic           ack;

    clocking cb @(posedge clk);
        default input #2ns output #3ns;
        output internal_clk;
    endclocking : cb
```



Coachip confidential

覆盖率 Coverage

```
program automatic test(busifc.TB ifc);
  class Transaction;
    rand bit [31:0] data;
    rand bit [ 2:0] port; // Eight port numbers
  endclass
  covergroup CovPort;
    coverpoint tr.port; // Measure coverage
  endgroup
  initial begin
    Transaction tr;
    CovPort ck;
    ck = new(); // Instantiate group
    tr = new();
    repeat (32) begin // Run a few cycles
    assert(tr.randomize); // Create a transaction
    ifc.cb.port <= tr.port; // and transmit
    ifc.cb.data <= tr.data; // onto interface
    ck.sample(); // Gather coverage
    @ifc.cb; // Wait a cycle
  end
endprogram
```

叩特电子Coachip

课后练习/问题

- SV与Verilog,Vera的关系？
- SV中为什么引入类

叩特电子Coachip

Q&A

昂特电子Coachip